

Instituto Tecnológico de Estudios Superiores de Monterrey



Evidencia 3. Artículo de investigación PBL3

Análisis de métodos de razonamiento e incertidumbre (Gpo 201)

Docente: Dr. Daniel Otero Fadul

Diego Armando Mijares Ledezma - **A01722421**

Pedro Soto Juárez - **A00837560**

Alexei Carrillo Acosta - **A01285424**

Marcos Renato Aquino Garcia - **A00835576**

Mauricio Octavio Valencia Gonzalez - **A01234397**

15 de octubre de 2024

Problematización

El problema de la localización de un robot en un entorno estático con Modelos Ocultos de Markov (HMM) se basa en la dificultad de determinar la posición exacta del robot cuando esta no es observable directamente. Aunque el robot puede moverse libremente entre las casillas vacías de una cuadrícula, su ubicación debe inferirse a partir de observaciones limitadas y propensas a errores provenientes de sensores que detectan obstáculos en direcciones cardinales (Norte, Sur, Este y Oeste). Este tipo de problema es crucial en robótica móvil, donde la incertidumbre del entorno y los errores en las mediciones juegan un rol fundamental en el cálculo de la posición. Como señala Rabiner (1989), los HMM son herramientas esenciales para modelar sistemas estocásticos donde las observaciones disponibles no revelan directamente el estado oculto subyacente del sistema, como es el caso de la localización en entornos no observables directamente.

Enfoque

El enfoque para resolver el problema de localización con HMM implica el uso de tres tipos de inferencias: filtrado, suavizado y el algoritmo de Viterbi. En primer lugar, el filtrado permite estimar la ubicación actual del robot basándose en las mediciones más recientes, mientras que el suavizado permite inferir una posición pasada utilizando tanto observaciones pasadas como futuras. El algoritmo de Viterbi, por otro lado, identifica la trayectoria más probable que el robot ha seguido, dadas las lecturas de los sensores. Según Murphy (2002), estas técnicas de inferencia permiten integrar las incertidumbres tanto en los movimientos del robot como en sus observaciones, lo que las convierte en una herramienta poderosa para la navegación robótica en entornos inciertos.

Propósito

El propósito de este proyecto es resolver un problema de localización a través del tiempo utilizando un proceso de Markov. Del proceso de Markov se utilizan un modelo de transición y un modelo del sensor cuyo objetivo es obtener la distribución de probabilidad conjunta de los estados y así poder obtener la localización actual del agente.

La actividad y sus recursos de apoyo están diseñados para servir como guía en la implementación de cada una de las partes de un HMM, desde los modelos anteriormente mencionados como las partes utilizadas para obtener las inferencias, así como las herramientas de visualización y sus respectivas interpretaciones.

Se usa el modelo de sensor para obtener una idea de en donde está ubicado el robot . Así mismo, se buscan obtener las inferencias básicas que se pueden realizar con un HMM. Se usa el filtrado para, de acuerdo a la información que se tiene, inferir la posición del robot en el tiempo actual. Para inferir su posición en un tiempo pasado se usa el *smoothing* o suavizado, mientras que para un tiempo futuro se usa la explicación más probable, todos utilizando la misma información que para hacer el filtrado. Para obtener la última inferencia también se busca implementar y comprender el algoritmo de Viterbi.

Los modelos ocultos de Markov tienen una alta variedad de aplicaciones, por lo que al finalizar este proyecto se espera una alta comprensión del modelo en general y de las diferentes partes que componen a un HMM en específico, para que de esta manera, haciendo los ajustes pertinentes, se pueda utilizar esta herramienta en diferentes proyectos de acuerdo a los diferentes propósitos que se le pueden dar a un HMM como reconocimiento de voz, reconocimiento de escritura a mano, traducción automática, entre otras.

Información

Teniendo en cuenta el problema, el enfoque y el propósito, se elaboró un código para resolver lo que se plantea. Primero, se importan las librerías que se requirieron: *numpy*, *matplotlib*, *numpy*, *seaborn*, *pandas*, y *itertools*. Siguiendo, se conlleva la creación de estados *S*, donde se mapea el mapa original de 16x4. Es decir, el diccionario *S* mapea cada estado a una posición 2D en el entorno discreto, donde cada estado corresponde a una celda identificada por coordenadas dentro del cuadro de 16x4. Con el entendimiento de que cada tupla (x,y) representa coordenadas del entorno en el estado, se puede observar que la utilidad de esto es representar la localización del robot.

Después, se define la función *neighbors_states*. Esta función construye un diccionario que asigna a cada estado una lista de sus estados vecinos, definidos como aquellos que están adyacentes en las cuatro direcciones cardinales (arriba, abajo, izquierda, derecha). La entrada de

la función incluye *states* (el diccionario de estados), y *location* (un booleano que indica si se devuelven vecinos como ubicaciones o índices de estado). La salida es un diccionario que asigna a cada estado una lista de sus vecinos. Para llegar a eso, la función recorre cada estado en el diccionario *states* y determina los vecinos válidos basándose en posibles movimientos en las direcciones cardinales.

Siguiente, se tiene la función *transition_matrix*, que genera una matriz de transición de tamaño $N \times N$ donde N es el número de estados. Cada elemento en la matriz representa la probabilidad de que el robot transite de un estado a otro. La entrada es el diccionario de estados S , y la salida la matriz de transición T . Por cada estado, se llena la fila correspondiente con probabilidades distribuidas uniformemente entre los estados vecinos.

La tercera función elaborada es *expected_measurements*. Esta función define las mediciones esperadas en cada estado del entorno. Las mediciones pueden ser cualquier propiedad del entorno, como características observables del espacio. La entrada a la función es el diccionario de estados S , y la salida un diccionario que asigna a cada estado una lista de mediciones esperadas. Para cada estado, se asigna una medición basada en un criterio (en este ejemplo, las coordenadas par/impar).

Sucesivo, se tiene la función *compute_discrepancy*, que calcula la discrepancia entre las mediciones reales observadas y las mediciones esperadas. Las entradas son las mediciones observadas y las mediciones esperadas para un estado, y la salida es el valor que representa la diferencia absoluta entre las mediciones observadas y esperadas. La función utiliza la norma $L1$ (suma de diferencias absolutas) para medir la discrepancia.

Posteriormente, se crea la función *observation_matrix*. Esta función construye una matriz de observación que relaciona los estados con las mediciones realizadas por el robot. Las entradas son el diccionario de estados S y mediciones. La salida es una matriz de observación O . Cada fila de la matriz representa un estado, y la columna correspondiente indica si la medición esperada se corresponde con la observada. Subsecuentemente, está la función *observation_matrices*, que genera un diccionario de matrices de observación para diferentes combinaciones de mediciones posibles. La entrada es el diccionario de estados S , y la salida es el diccionario de matrices de observación. Para cada posible combinación de mediciones, se crea una matriz de observación y se almacena en el diccionario. Después, se define *epsilon* (probabilidad de error de medición) como 0.1 y se le asigna a *observation_matrices* como O .

Siguiente, se tiene la función *robot_measurements*, que simula las mediciones realizadas por el robot en su entorno, dado su estado actual. Entra el estado actual del robot y el diccionario de estados S , y sale una lista de mediciones correspondientes a ese estado. La función calcula mediciones sencillas basadas en las coordenadas del estado actual. Después, para simular la caminata del robot, se tiene la función *robot_random_walk*, que simula un paseo aleatorio del robot por el entorno, seleccionando un estado vecino al azar en cada paso. De entrada a la función se tiene el diccionario de estados S y el número de pasos. De salida se obtiene una lista de las posiciones que el robot visitó durante su caminata aleatoria. En cada paso, el robot elige aleatoriamente uno de los estados vecinos y se mueve a ese estado.

Ahora sí, se crean las siguientes variables importantes que inicializan el estado del sistema y registran las observaciones del robot. $X0$ es el vector de probabilidades iniciales, asumiendo una distribución uniforme sobre todos los estados. $Path$ es el camino aleatorio recorrido por el robot. E son las mediciones realizadas por el robot a lo largo del camino. También se crea la función de *filtering*. Se utiliza para calcular la distribución de probabilidad sobre los estados en un modelo oculto de Markov (Hidden Markov Model, HMM) dado un conjunto de observaciones, avanzando en el tiempo a medida que se reciben nuevas observaciones. El filtrado solo toma en cuenta las observaciones hasta el momento actual.

Siguiente, para la visualización, se tiene la función *view_heatmap* que genera un mapa de calor para visualizar las distribuciones de probabilidad o los estados del robot - con entrada de *matrix* y S . Después, se visualiza el mapa de calor de la matriz de transición y se imprime el *path*. Siguiente, se establece la función de *smoothing*. Aquella implementa el algoritmo de suavizado (o filtrado hacia atrás) basado en la inferencia de estados ocultos. Este tipo de algoritmo se utiliza en modelos de Markov ocultos para calcular la probabilidad posterior de una secuencia de estados dados una serie de observaciones. El objetivo del suavizado es obtener la distribución de probabilidad de cada estado en la secuencia, considerando todas las observaciones pasadas y futuras. Esto permite tener una mejor estimación de los estados que tuvo el sistema en el pasado, dado el conocimiento de las observaciones posteriores. Las entradas de la función son... E : Lista de mediciones observadas en cada paso del tiempo. O : Diccionario de matrices de observación que contienen las probabilidades de realizar una medición dada la posición del robot. S : El diccionario de estados. T : Matriz de transición. $X0$: Vector de probabilidades iniciales. De salida se tiene, X , la matriz que contiene las probabilidades suavizadas para cada estado en cada paso de tiempo. La matriz tiene dimensiones $T \times N$, donde T es el número de observaciones. Con esto, posteriormente se visualiza el heatmap con las probabilidades suavizadas, y se imprime el *path*.

Por último, se tiene la función *most_likely_sequence*, que implementa el algoritmo de Viterbi, utilizado para encontrar la secuencia más probable de estados ocultos en un modelo

oculto de Markov (HMM), dado un conjunto de observaciones. A diferencia del algoritmo de suavizado, que calcula distribuciones de probabilidad sobre los estados en cada tiempo, este algoritmo busca determinar una única secuencia de estados que maximice la probabilidad conjunta de la secuencia observada y los estados ocultos. Maneja las mismas entradas que la función de suavizado, y la salida es únicamente del *path*. Ya con el camino impreso del camino más probable, se puede comparar con la secuencia genuina del robot. Por último, se imprime el *heatmap* de la ubicación del robot..

Resultados

Por medio de la función *random robot walk* se simula como se mueve un robot en una cuadrícula, *path* es el camino que ha tomado el robot en la cuadrícula y la evidencia obtenida es recibida por los sensores del robot en cada paso en binario.

Simulación a largo de 10 pasos (<i>Filtering</i>)	
Path	[(5, 3), (5, 2), (5, 3), (5, 2), (5, 3), (5, 2), (5, 1), (5, 0), (6, 0), (7, 0)]
Evidence	['1010', '0011', '1010', '0011', '1011', '0011', '0011', '0001', '1100', '1100']

En *path*, significa que el robot comenzó en la posición (5, 3), luego se movió a (5, 2), regresó a (5, 3), y así sucesivamente, hasta que terminó en la posición (7, 0). Mientras tanto en cada número binario son las cuatro direcciones (Norte, Este, Sur, Oeste) durante su movimiento por la cuadrícula.

El robot se mueve en una cuadrícula de 16 filas por 4 columnas. Para estimar dónde está el robot, usamos un vector de probabilidades que tiene 42 componentes, cada una correspondiente a una posición válida en la cuadrícula. Cada vez que el robot recibe una nueva medición, el algoritmo de filtrado actualiza las probabilidades.

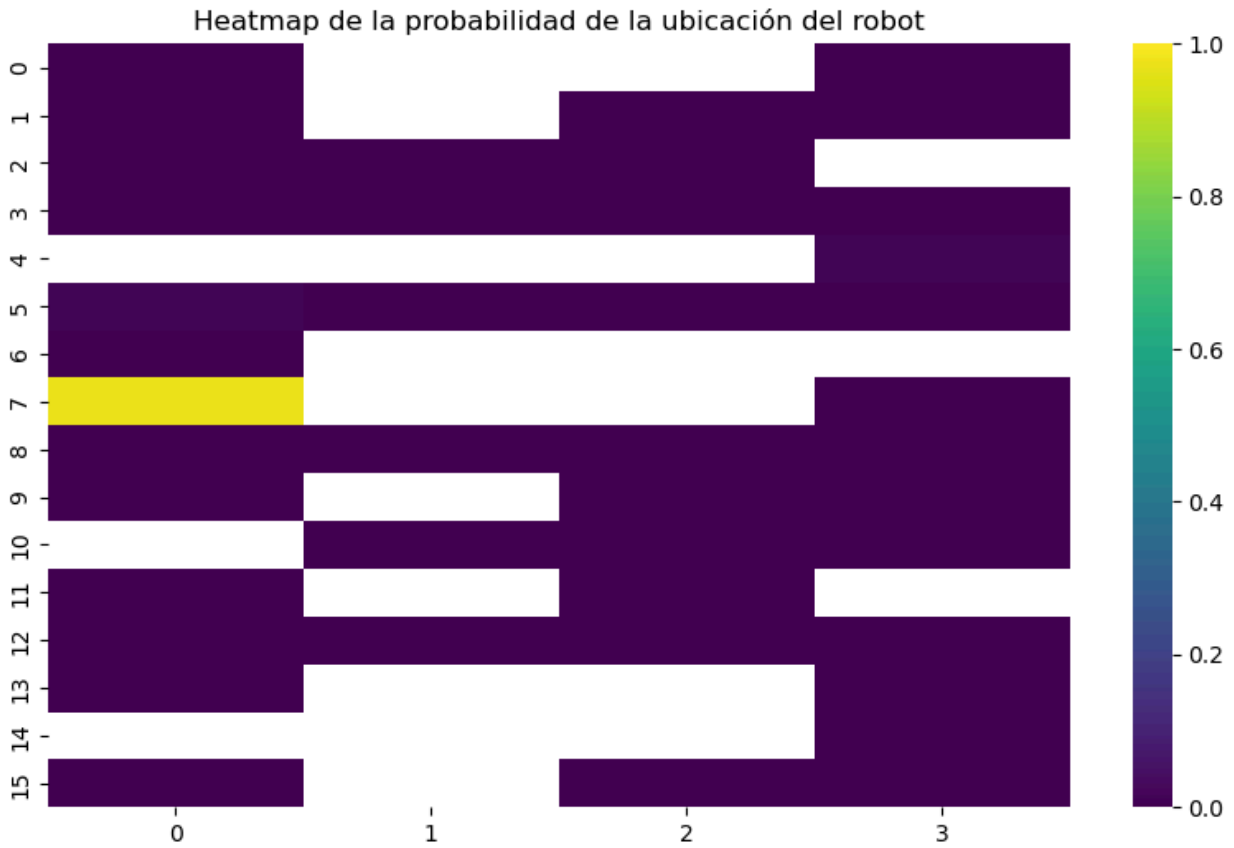
Probabilidades de observar cada paso					
Fila	Columna	Probabilidad	Fila	Columna	Probabilidad
0	0	0.00011979	4	2	0.00143746
0	1	0.00287491	4	3	2.2183E-05
0	2	0.00031943	5	0	0.02263995
0	3	0.00143746	5	1	0.02263995
1	0	2.66E-05	5	2	0.00215619
1	1	0.00215619	5	3	0.00027951
1	2	0.00179682	6	0	0.00021961
1	3	0.00047915	6	1	0.0050311
2	0	0.00646856	6	2	0.00125778
2	1	7.99E-05	6	3	0.02910851
2	2	0.00059894	7	0	0.00011979
2	3	0.00646856	7	1	0.00011979
3	0	0	7	2	0.26197659
3	1	0.00023958	7	3	0.00179682
3	2	0.02263995	8	0	0.00215619
3	3	0.00215619	8	1	0.17465106
4	0	0.00143746	8	2	7.9859E-05
4	1	5.99E-05	8	3	0.03234279
4	2	0.00143746	9	0	0.02263995

4	3	2.22E-05	9	1	0.10187979
5	0	0.02263995	9	2	0.00179682
5	1	0.02263995	9	3	0.00215619
5	2	0.00215619	10	0	0.00215619
5	3	0.00027951	10	1	0.26197659

La tabla representa las probabilidades de observar cada paso , al moverse el robot agarra el que tiene mayor probabilidad que ocurra un movimiento con respecto a otros y así sucesivamente se generan otras probabilidades al hacer otros pasos hasta llegar al (7,0). La idea central es que el robot siempre selecciona el paso más probable, basándose en las probabilidades dadas. El objetivo de la tabla es mostrar esas probabilidades para cada posición en la cuadrícula, no necesariamente el orden exacto en el que el robot visitará dichas posiciones.

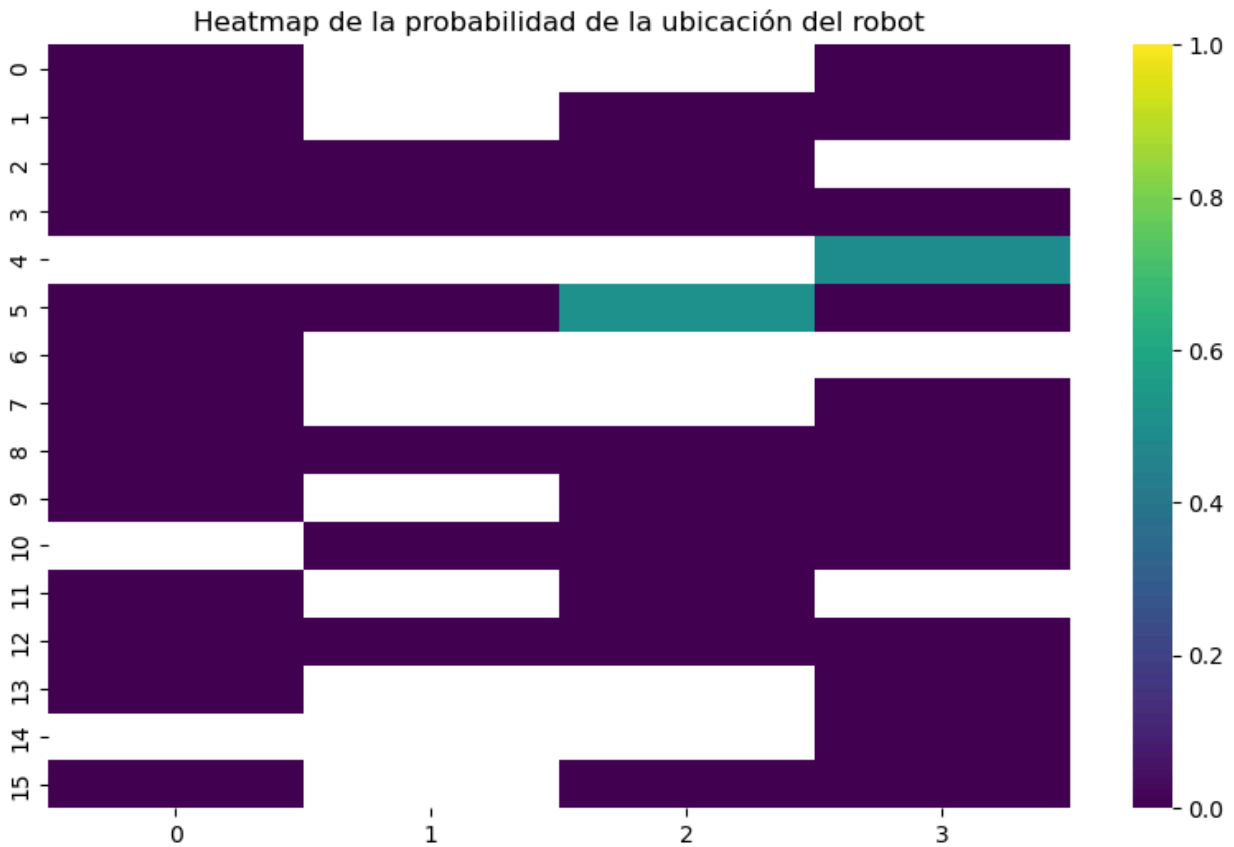
Posteriormente, se tiene la visualización del camino recorrido por el robot, representada en mapas de calor. De esta forma se tiene dicho recorrido para las inferencias que fueron obtenidas: Filtrado, smoothing y secuencia más probable. El mapa de calor representa las ubicaciones más probables donde puede estar el robot, por eso se puede observar el recorrido, pero al estar representando realmente las ubicaciones, la última ubicación del robot, es la ubicación más probable y por lo tanto, la que tiene un valor mayor, representado en el mapa de calor con un color más brillante y cercano al amarillo.

Mapa de calor de Filtrado:



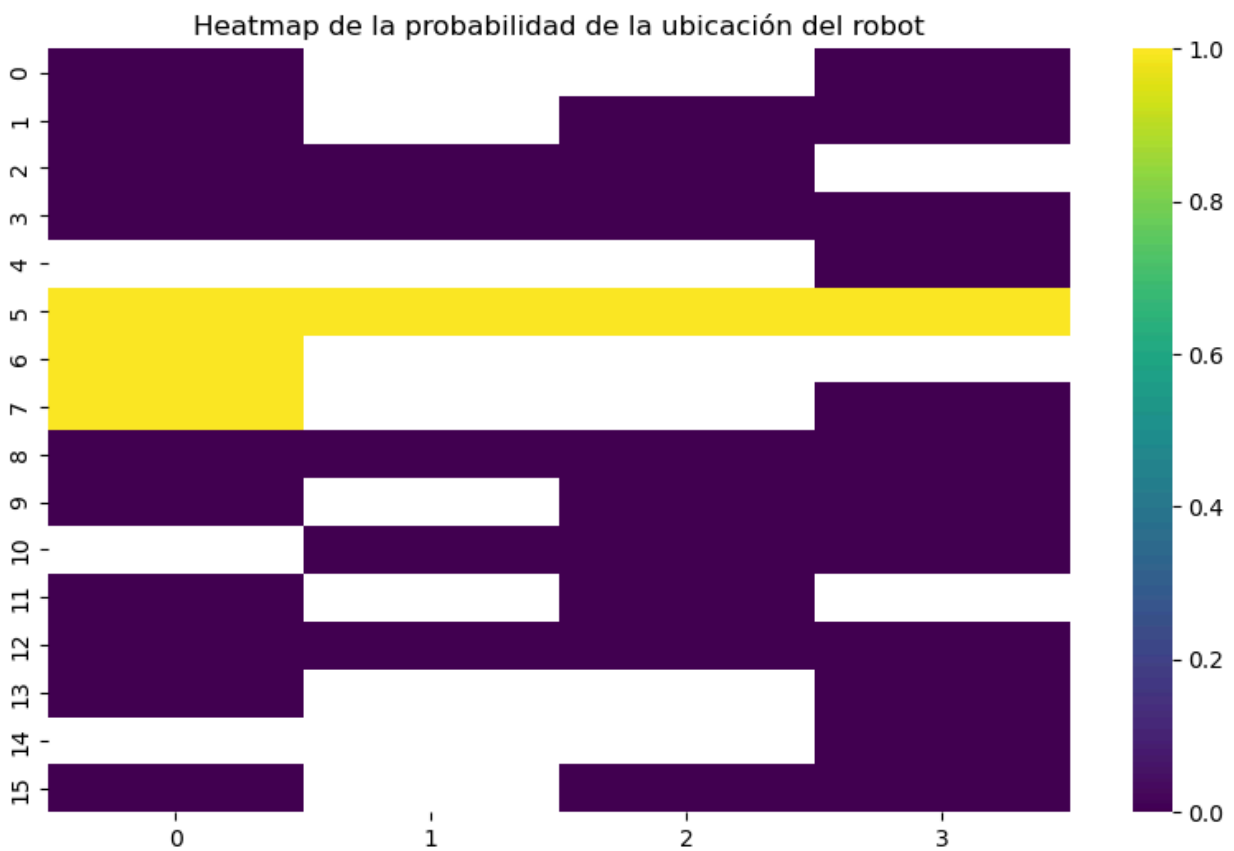
Este heatmap muestra la probabilidad de ubicación del robot en una cuadrícula de 16x4, donde cada celda representa una posible posición del robot, y los colores indican la magnitud de la probabilidad asociada a esa posición, donde en cada paso se toman diferentes probabilidades.

Mapa de calor de Smoothing:



Este heatmap muestra la probabilidad de que el robot se encuentre en las celdas mostradas. En este caso, con los datos actuales, se obtiene la probabilidad de la ubicación del robot, pero en estados pasados de tiempo.

Mapa de calor de secuencia más probable:



Este heatmap, al igual que los otros 2 utiliza la información actual para obtener las probabilidades de la ubicación del robot, pero a diferencia de estos, la utiliza para predecir la ubicación del robot en estados futuros.

En esta ocasión, el *path* que se pudo observar en *filtering* se mantiene:

<i>Path y evidencia de most likely sequence</i>
$(5,3)$
$(5,2)$
$(5,3)$
$(5,2)$
$(5,1)$
$(5,2)$
$(5,1)$
$(5,0)$
$(6,0)$
$(7,0)$

Se analiza a más detalle el camino: $(5, 3)$ y $(5, 2)$ son las posiciones más recurrentes en la lista, lo que indica que el robot ha estado moviéndose entre estas dos posiciones con alta probabilidad. Luego, el robot se desplaza a $(5, 1)$ y posteriormente a $(5, 0)$, lo que sugiere que el robot ha cambiado de área, posiblemente hacia un borde de la cuadrícula. Finalmente, el robot llega a $(6, 0)$ y hasta $(7,0)$.

Por último, se imprime el *path* del robot, que imprime casi mismo camino, reafirmando el cercano acierto de la función de *most likely path*. Se muestra el camino genuino, con la diferencia marcada.

<i>Path y evidencia de most likely sequence</i>
$(5,3)$
$(5,2)$

(5,3)
(5,2)
<u>(5,3)</u> (diferencia)
(5,2)
(5,1)
(5,0)
(6,0)
(7,0)

Conclusiones

Con estos resultados, se logró obtener un modelo el cual con cadenas ocultas de Markov se simula el movimiento de un robot dentro de un sistema discreto que se representa en una cuadrícula de 42 posibles estados válidos en los cuales el robot puede posicionarse y moverse entre ellos. En el desarrollo de este proyecto, se implementaron distintas funciones claves para lograr la simulación de movimiento y la estimación de probabilidad de dónde puede ir el robot con técnicas como el suavizado y el filtrado. Esta simulación incluye la generación del heat map que permite la visualización del recorrido del robot y las probabilidades asociadas a su ubicación. Los heatmaps de igual forma sirvieron para visualizar que hace cada tipo de técnica. Al experimentar con los steps se pudo observar como el robot se mueve ya con más iteraciones, lo que puede generar que el robot se quede en un ciclo.

Algunos de los logros más importantes que se lograron fue poder tener una simulación eficiente, poder estimar los estados y poder visualizar el recorrido. Con esto, también se tuvieron ciertos retos al realizar el proyecto, como lo fue asegurar que el sistema de coordenadas estuvieran correctamente alineados para que el robot pueda moverse de manera realista a su alrededor.

La inferencia probabilística aplicada en el proyecto permite observar el camino predicho por los algoritmos de filtrado y suavizado, el cual se pudo observar como el camino más probable coincide con el recorrido real del robot, lo que refuerza la precisión de las técnicas en

estimar la posición. Este hallazgo sirvió para entender la significancia que con base a mediciones ruidosas, aún es posible realizar predicciones confiables sobre la trayectoria de un sistema dinámico. Además, el número de pasos influye directamente en la certeza de la predicción, a mayor cantidad de pasos, más información se tiene y mejor llega a ser la inferencia. Por otro lado, la tasa de error afecta la calidad de las mediciones, mientras exista un error rate mayor, más ruido se introduce en los datos lo que puede llevar a predicciones menos precisas. Estos elementos se combinan para crear una comprensión del uso de estas técnicas, lo que puede ayudar a transformar la toma de decisiones en entornos inciertos, al reducir la incertidumbre y mejorar la capacidad del sistema

En resumen, en este proyecto se logró integrar métodos de simulación, inferencia probabilística y visualización para modelar el movimiento de un robot en un espacio discreto. La combinación de todo esto, también nos permite hacer inferencias sobre su ubicación, proporcionando una visión clara de cómo se mueve y que áreas a recorrido.

Referencias

- Rabiner, L. (1989). A tutorial on hidden Markov models and selected applications in speech recognition.
- Murphy, K. (2002). Dynamic Bayesian Networks: Representation, Inference and Learning